

Einsendeaufgabe DEV-E2: Supported AI and GenAI Coding – Pet Project

Smart Task Manager

Entwicklung einer AI-unterstützten Webanwendung

Aufgabenstellung

Im Rahmen dieser Einsendeaufgabe sollte ein eigenes Pet Project unter Einsatz moderner AI- und GenAI-Werkzeuge entwickelt werden. Ziel war die Umsetzung eines funktionsfähigen Software-Prototyps, dessen Planung, Entwicklung und Dokumentation nachvollziehbar dargestellt werden.

Neben der eigentlichen Implementierung sollte gezeigt werden, wie AI-Werkzeuge sinnvoll in den Softwareentwicklungsprozess integriert werden können. Dazu gehörten sowohl die Planung der Anwendung als auch die schrittweise Umsetzung sowie die Dokumentation der einzelnen Entwicklungsschritte.

Projektidee

Als Projekt wurde ein **Smart Task Manager** entwickelt. Die Anwendung unterstützt Benutzer bei der Verwaltung persönlicher Aufgaben, Prioritäten und Bearbeitungsstände. Ziel ist eine übersichtliche und leicht bedienbare Webanwendung, welche den aktuellen Bearbeitungsstatus einzelner Aufgaben jederzeit sichtbar macht.

Die Anwendung baut direkt auf dem im Rahmen von DEV-E1 entwickelten GUI-Prototyp auf. Während in DEV-E1 ausschließlich die Benutzeroberfläche entworfen wurde, erfolgt in DEV-E2 die technische Umsetzung der Anwendung als Webprojekt mit Frontend, Backend und einer REST-Schnittstelle.

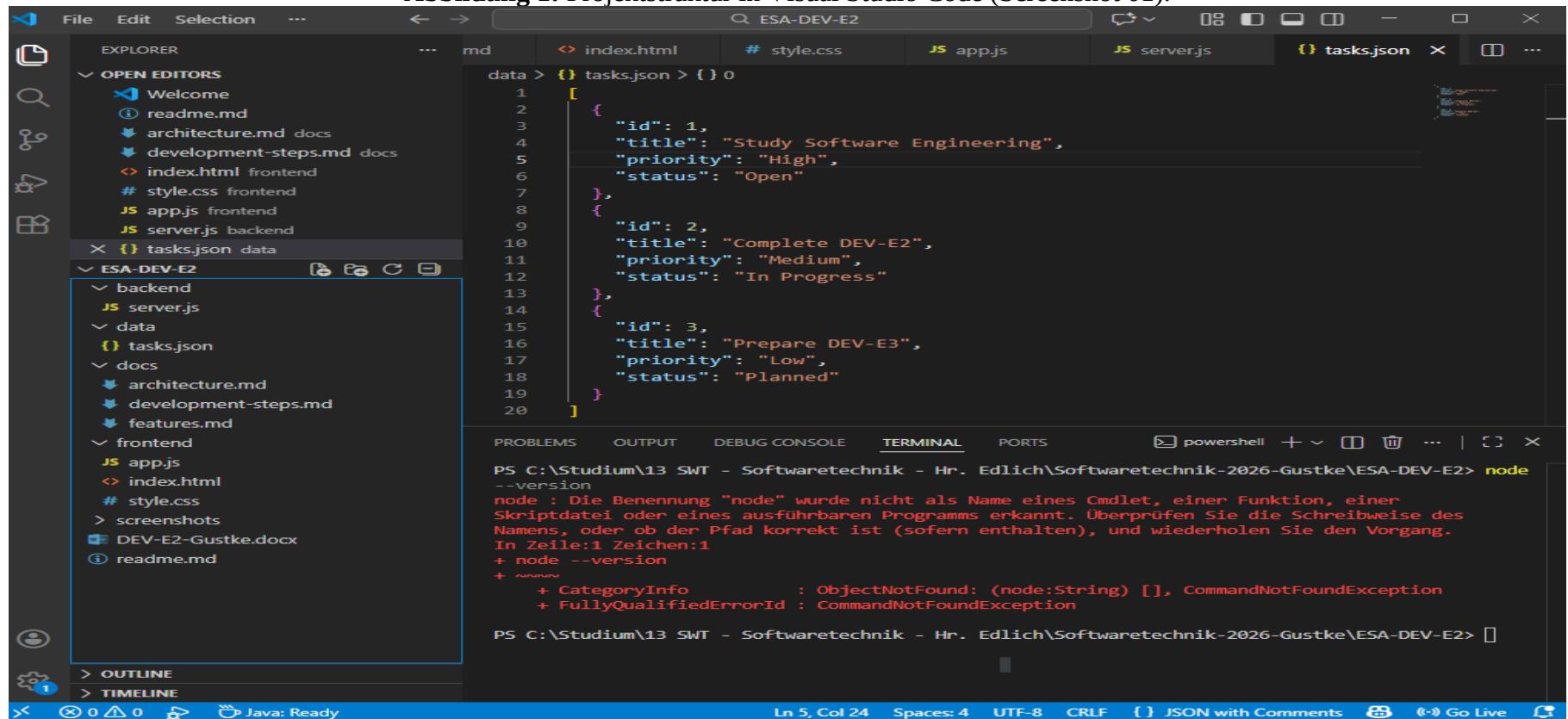
Dadurch entsteht eine modulare Architektur, welche gleichzeitig die Grundlage für DEV-E3 bildet. Dort wird die Anwendung zu einer verteilten Software erweitert

Projektstruktur

Die folgende Ordnerstruktur wurde gewählt, um Frontend, Backend, Datenhaltung und Dokumentation klar voneinander zu trennen.

- **frontend** enthält die Benutzeroberfläche mit HTML, CSS und JavaScript.
- **backend** enthält den Server auf Basis von Node.js und Express.
- **data** enthält die JSON-Datei mit den Aufgaben.
- **docs** enthält die technische Projektdokumentation im Markdown-Format.
- **screenshots** dokumentiert die einzelnen Entwicklungsschritte.

Abbildung 1: Projektstruktur in Visual Studio Code (Screenshot 01).



Implementierung

Die Entwicklung erfolgte schrittweise. Zunächst wurde das Frontend erstellt. Anschließend wurde ein Backend mit Node.js und Express entwickelt. Beide Komponenten kommunizieren über eine REST-Schnittstelle miteinander.

Die Aufgaben werden in einer JSON-Datei gespeichert und vom Backend eingelesen. Das Frontend ruft diese Daten über HTTP-Anfragen ab und stellt sie anschließend innerhalb der Benutzeroberfläche dar.

Durch diese Trennung zwischen Benutzeroberfläche, Serverlogik und Datenhaltung entstand bereits eine klar strukturierte und modular aufgebaute Anwendung.

Frontend

Das Frontend wurde mit den Webtechnologien HTML, CSS und JavaScript umgesetzt. HTML bildet die Struktur der Benutzeroberfläche, CSS sorgt für das Layout und das Erscheinungsbild der Anwendung, während JavaScript die Programmlogik im Browser übernimmt.

Die Benutzeroberfläche ermöglicht die Anzeige der vorhandenen Aufgaben sowie die Eingabe neuer Aufgaben. Die eigentlichen Aufgabendaten werden jedoch nicht lokal im Browser gespeichert, sondern vom Backend über eine REST-API geladen. Dadurch bleiben Benutzeroberfläche und Datenverarbeitung klar voneinander getrennt.

Abbildung 2: Benutzeroberfläche des Smart Task Managers (Screenshot 07).

The screenshot shows a web browser window with the address bar displaying a file path. The application has a blue header with the title "Smart Task Manager" and the tagline "Manage your daily tasks efficiently". Below the header, there are two main sections: "Create Task" and "My Tasks".

Create Task

Task title

Low

Add Task

My Tasks

Study Software Engineering Priority: High Status: Open
Complete DEV-E2 Priority: Medium Status: In Progress
Prepare DEV-E3 Priority: Low Status: Planned

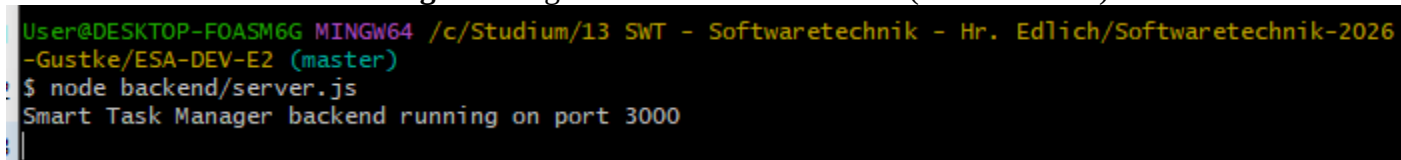
Backend

Das Backend wurde mit **Node.js** und dem Webframework **Express** umgesetzt. Es stellt eine REST-Schnittstelle bereit, über die das Frontend auf die gespeicherten Aufgaben zugreifen kann.

Beim Start des Servers wird auf Port 3000 ein Webserver gestartet. Anfragen an den Endpunkt `/tasks` werden verarbeitet, indem die Daten aus der Datei `tasks.json` eingelesen und als JSON-Antwort an das Frontend zurückgegeben werden.

Durch diese Trennung von Benutzeroberfläche und Serverlogik entstand eine klar strukturierte Anwendung, deren einzelne Komponenten unabhängig voneinander erweitert werden können.

Abbildung 3: Erfolgreicher Start des Backends (Screenshot 05).



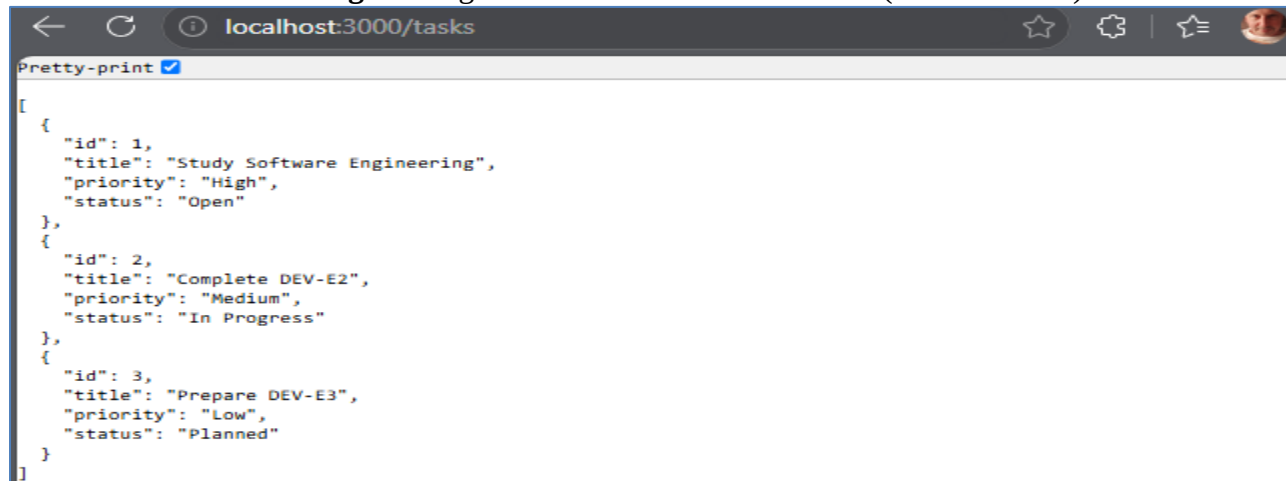
```
User@DESKTOP-FOASM6G MINGW64 /c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026
-Gustke/ESA-DEV-E2 (master)
$ node backend/server.js
Smart Task Manager backend running on port 3000
```

REST-API und Datenhaltung

Für den Datenaustausch zwischen Frontend und Backend wurde eine REST-Schnittstelle implementiert. Das Frontend sendet eine HTTP-Anfrage an den Endpunkt `http://localhost:3000/tasks`. Das Backend liest daraufhin die Aufgaben aus der Datei `tasks.json` und liefert diese im JSON-Format zurück.

Die Speicherung der Daten in einer separaten JSON-Datei ermöglicht eine einfache Trennung zwischen Datenhaltung und Programmlogik. Gleichzeitig bildet diese Struktur eine geeignete Grundlage für spätere Erweiterungen, beispielsweise durch den Einsatz einer Datenbank.

Abbildung 4: Ausgabe der REST-API im Browser (Screenshot 06).



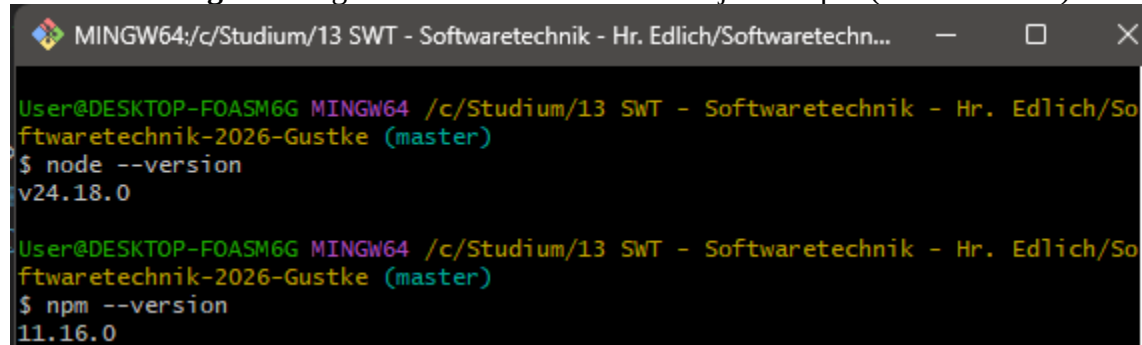
```
[
  {
    "id": 1,
    "title": "Study Software Engineering",
    "priority": "High",
    "status": "Open"
  },
  {
    "id": 2,
    "title": "Complete DEV-E2",
    "priority": "Medium",
    "status": "In Progress"
  },
  {
    "id": 3,
    "title": "Prepare DEV-E3",
    "priority": "Low",
    "status": "Planned"
  }
]
```

Test der Anwendung

Nach Abschluss der Implementierung wurde die Anwendung erfolgreich getestet. Zunächst wurden Node.js und die benötigten Bibliotheken installiert. Anschließend wurde das Backend gestartet und die REST-Schnittstelle im Browser überprüft.

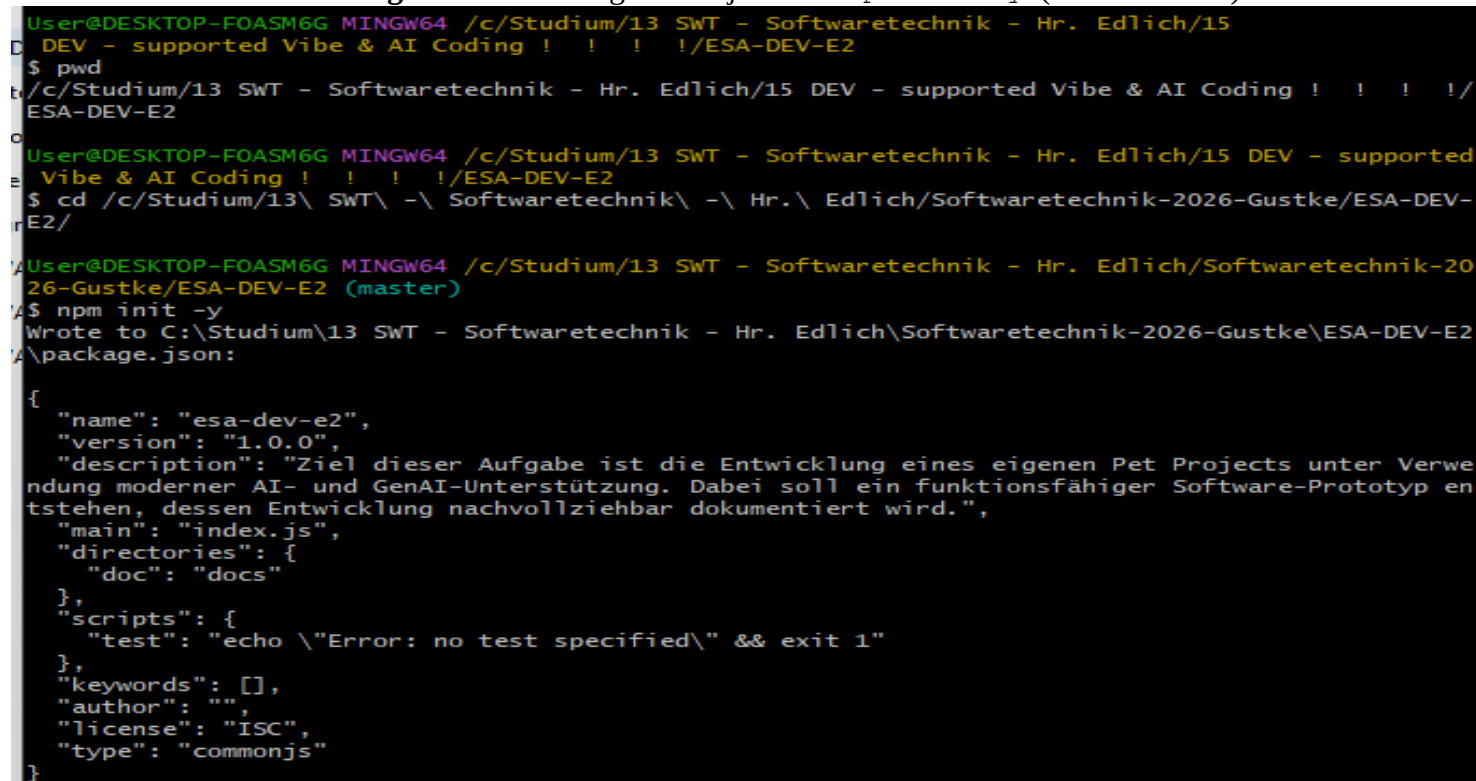
Im letzten Schritt wurde die Datei `index.html` geöffnet. Das Frontend konnte die vom Backend bereitgestellten Aufgaben erfolgreich laden und innerhalb der Benutzeroberfläche darstellen. Damit konnte die Kommunikation zwischen Frontend und Backend erfolgreich nachgewiesen werden.

Abbildung 5: Erfolgreiche Installation von Node.js und npm (Screenshot 02).



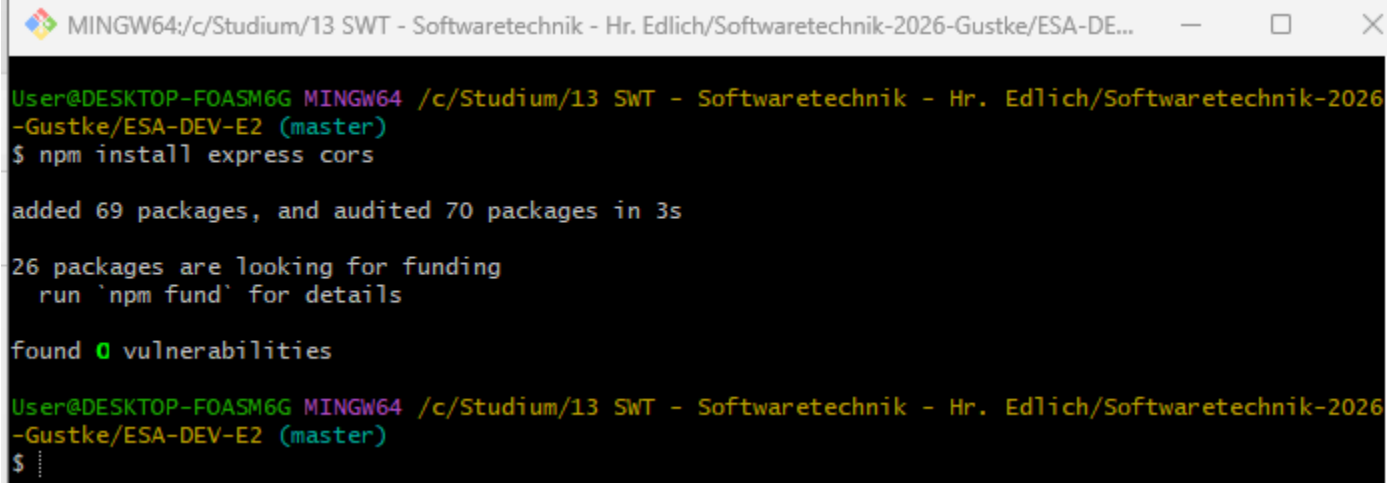
```
MINGW64:/c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechn...
User@DESKTOP-FOASM6G MINGW64 /c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke (master)
$ node --version
v24.18.0
User@DESKTOP-FOASM6G MINGW64 /c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke (master)
$ npm --version
11.16.0
```

Abbildung 6: Initialisierung des Projekts mit npm init -y (Screenshot 03).



```
User@DESKTOP-FOASM6G MINGW64 /c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/15 DEV - supported Vibe & AI Coding ! ! ! !/ESA-DEV-E2
$ pwd
/c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/15 DEV - supported Vibe & AI Coding ! ! ! !/ESA-DEV-E2
User@DESKTOP-FOASM6G MINGW64 /c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/15 DEV - supported Vibe & AI Coding ! ! ! !/ESA-DEV-E2
$ cd /c/Studium/13\ SWT\ -\ Softwaretechnik\ -\ Hr.\ Edlich\Softwaretechnik-2026-Gustke\ESA-DEV-E2/
User@DESKTOP-FOASM6G MINGW64 /c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-DEV-E2 (master)
$ npm init -y
Wrote to C:\Studium\13 SWT - Softwaretechnik - Hr. Edlich\Softwaretechnik-2026-Gustke\ESA-DEV-E2\package.json:
{
  "name": "esa-dev-e2",
  "version": "1.0.0",
  "description": "Ziel dieser Aufgabe ist die Entwicklung eines eigenen Pet Projects unter Verwendung moderner AI- und GenAI-Unterstützung. Dabei soll ein funktionsfähiger Software-Prototyp entstehen, dessen Entwicklung nachvollziehbar dokumentiert wird.",
  "main": "index.js",
  "directories": {
    "doc": "docs"
  },
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Abbildung 7: Installation der benötigten Bibliotheken Express und CORS (Screenshot 04).



```
MINGW64:/c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-DE...
User@DESKTOP-FOASM6G MINGW64 /c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-DEV-E2 (master)
$ npm install express cors

added 69 packages, and audited 70 packages in 3s

26 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

User@DESKTOP-FOASM6G MINGW64 /c/Studium/13 SWT - Softwaretechnik - Hr. Edlich/Softwaretechnik-2026-Gustke/ESA-DEV-E2 (master)
$
```

AI- und GenAI-Unterstützung

Während der Entwicklung wurden verschiedene AI- und GenAI-Werkzeuge unterstützend eingesetzt. Diese halfen insbesondere bei der Strukturierung des Projekts, der Entwicklung der Softwarearchitektur, der Erstellung von Beispielcode sowie bei der Dokumentation der einzelnen Entwicklungsschritte.

Alle erzeugten Inhalte wurden fachlich überprüft, angepasst und getestet. Die AI diente somit als Entwicklungsunterstützung, während die Planung, Bewertung und Integration der einzelnen Komponenten eigenständig erfolgte.

Zusätzlich wurde AI zur Erläuterung von Programmcode, zur Kommentierung sowie zur schrittweisen Entwicklung und Fehleranalyse eingesetzt. Alle erzeugten Ergebnisse wurden manuell überprüft, getestet und bei Bedarf angepasst.

Fazit

Im Rahmen dieser Einsendeaufgabe wurde ein funktionsfähiger Prototyp eines Smart Task Managers entwickelt. Die Anwendung besteht aus einem Frontend, einem Backend sowie einer einfachen JSON-basierten Datenhaltung. Die Kommunikation zwischen den Komponenten erfolgt über eine REST-Schnittstelle.

Durch den gezielten Einsatz von AI- und GenAI-Werkzeugen konnte der Entwicklungsprozess beschleunigt und strukturiert unterstützt werden. Gleichzeitig zeigte sich, dass ein grundlegendes Verständnis der Softwarearchitektur und eine fachliche Überprüfung aller Ergebnisse weiterhin notwendig sind.

Die entstandene Anwendung bildet eine geeignete Grundlage für die folgende Einsendeaufgabe DEV-E3, in der das Projekt zu einer echten verteilten Anwendung erweitert wird.

Die entwickelte Anwendung erfüllt die Anforderungen der Einsendeaufgabe DEV-E2 vollständig und bildet gleichzeitig die technische Grundlage für DEV-E3. Dort wird die bisherige Client-Server-Anwendung zu einer verteilten Anwendung mit mehreren miteinander kommunizierenden Komponenten erweitert.